Министерство образования Новосибирской области

ГБПОУ НСО «Новосибирский авиационный технический колледж имени Б.С. Галущака»

ОТЧЕТ

по лабораторной работе № 3

Условный оператор if

Вариант 2

Выполнил: студент группы ПР-25.106

Ф.И.О. Ушаков Ю.С.

Новосибирск 2026

СОДЕРЖАНИЕ

1 Цель и задачи работы. Теоретические сведения 3

2 Задание 1. Вычисление выражения 4

Блок-схема задания 1 5

3 Задание 2. Принадлежность точки области 6

Блок-схема задания 2 7

4 Задание 3. Определение вида треугольника 8

Блок-схема задания 3 9

Тестирование задания 3 10

5 Ответы на контрольные вопросы. Вывод 11

1 ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Цель работы - изучить условный оператор if в Python и научиться составлять программы с выбором действий в зависимости от введенных данных.

Задачи работы: изучить формы if, if-else и if-elif-else; составить блок-схемы; написать три программы второго варианта; проверить результаты на разных исходных данных.

Общие теоретические сведения

Условный оператор проверяет выражение, которое принимает значение True или False. Если условие после if истинно, выполняется соответствующий блок команд. В противном случае проверяется elif или выполняется блок else.

Блоки кода выделяются отступами. После условия и после else ставится двоеточие. Для сравнения используются ==, !=, <, >, <=, >=. Оператор and требует выполнения всех условий, or - хотя бы одного, not меняет логическое значение на противоположное.

Функция input() получает строку с клавиатуры, float() преобразует ее в вещественное число, print() выводит результат. Для работы с синусом и числом π используется модуль math. Аргумент sin() задается в радианах.

В работе используются отдельные файлы zadanie1.py, zadanie2.py и zadanie3.py. Для запуска достаточно открыть нужный файл в Visual Studio Code и запустить его, затем ввести значения в терминале. Дробные числа вводятся через точку.

2 ЗАДАНИЕ 1

Условие: запрограммировать выражение (a + b - f / a) + f · a · a - (a + b). Числа a, b и f вводятся с клавиатуры. Подсказки должны показывать порядок ввода.

Описание алгоритма

Вводятся три числа. Если a равно нулю, выводится сообщение об ошибке, поскольку деление на ноль невозможно. В остальных случаях вычисляется выражение и выводится результат.

Исходный код

```
# Вычисление выражения
a = float(input("Введите a: "))
b = float(input("Введите b: "))
f = float(input("Введите f: "))

if a == 0:
    print("Ошибка: a не должно быть равно нулю")
else:
    result = (a + b - f / a) + f * a * a - (a + b)
    print("Результат:", result)
```

Тестирование и результаты

a; b; f	Ожидаемый результат	Полученный результат
2; 3; 4	14	14.0
1; 7; 5	0	0.0
-2; 3; 4	18	18.0
0; 3; 4	Сообщение об ошибке	a не должно быть равно нулю

При a = 2, b = 3, f = 4 получено: (2 + 3 - 4 / 2) + 4 · 2 · 2 - (2 + 3) = 14. Результат программы совпадает с ручным расчетом.

БЛОК-СХЕМА ЗАДАНИЯ 1

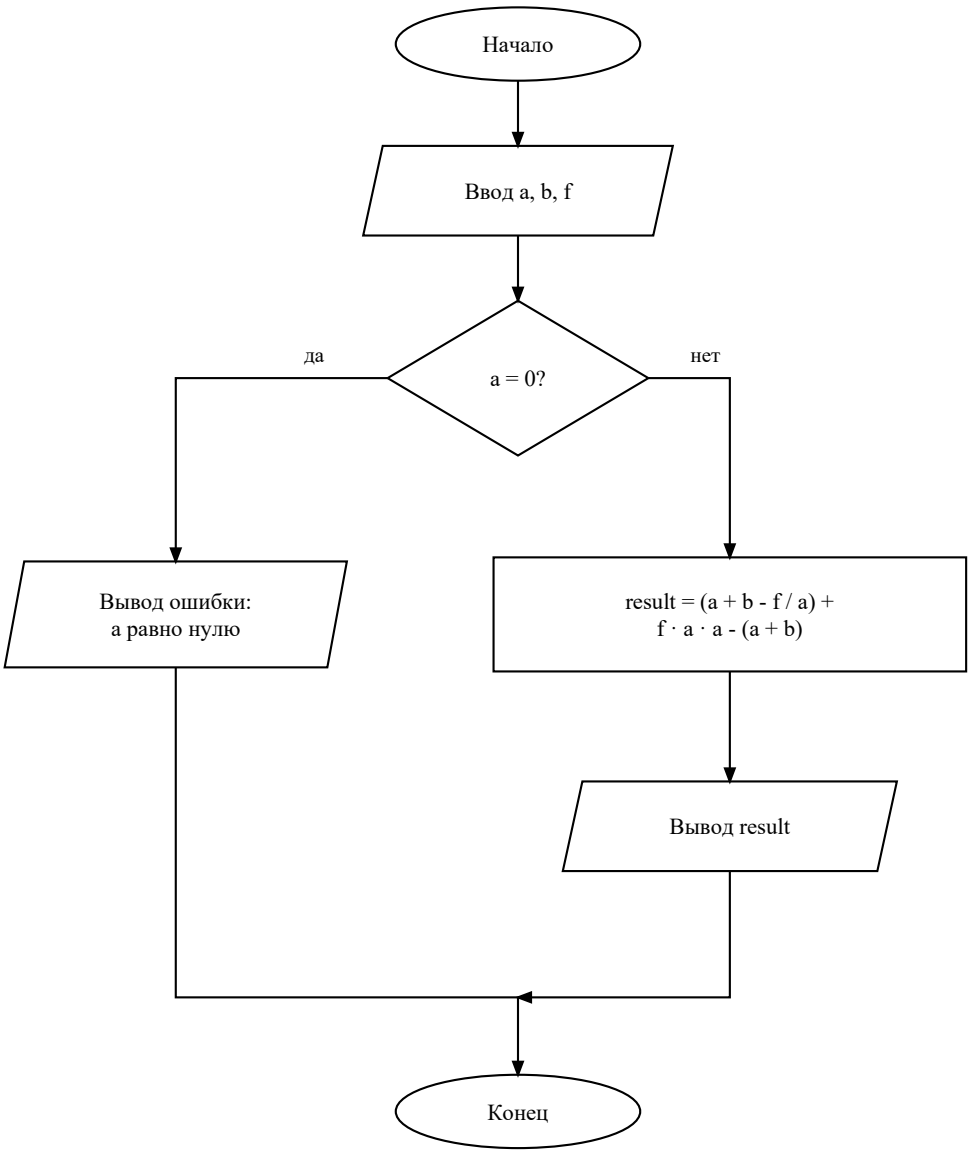


Рисунок 1 - Блок-схема вычисления выражения

3 ЗАДАНИЕ 2

Условие: определить, принадлежит ли точка с координатами (x; y) заштрихованной области на рисунке варианта 2.

Описание алгоритма

Область находится над осью Ox, под прямой y = 0,5 и под первой положительной полуволной графика y = sin(x). Поэтому одновременно проверяются условия: $0 \le x \le \pi$, $0 \le y \le 0{,}5$ и $y \le \sin(x)$. Границы включаются в область.

Значения x и y вводятся с клавиатуры. Если все условия выполняются, выводится сообщение о принадлежности точки области. Иначе выводится отрицательный ответ. Добавка 0,000000001 учитывает небольшую погрешность вычисления синуса на границе.

Исходный код

```
from math import sin, pi

# Проверка попадания точки в заштрихованную область
x = float(input("Введите x: "))
y = float(input("Введите y: "))

if 0 <= x <= pi and 0 <= y <= 0.5 and y <= sin(x) + 0.000000001:
    print("Точка принадлежит области")
else:
    print("Точка не принадлежит области")
```

Тестирование и результаты

x; y	Ожидаемый результат	Полученный результат
1; 0.3	Принадлежит	Принадлежит
1; 0.6	Не принадлежит	Не принадлежит
0.1; 0.4	Не принадлежит	Не принадлежит
-1; 0	Не принадлежит	Не принадлежит
0; 0	Принадлежит	Принадлежит
π; 0	Принадлежит	Принадлежит

БЛОК-СХЕМА ЗАДАНИЯ 2

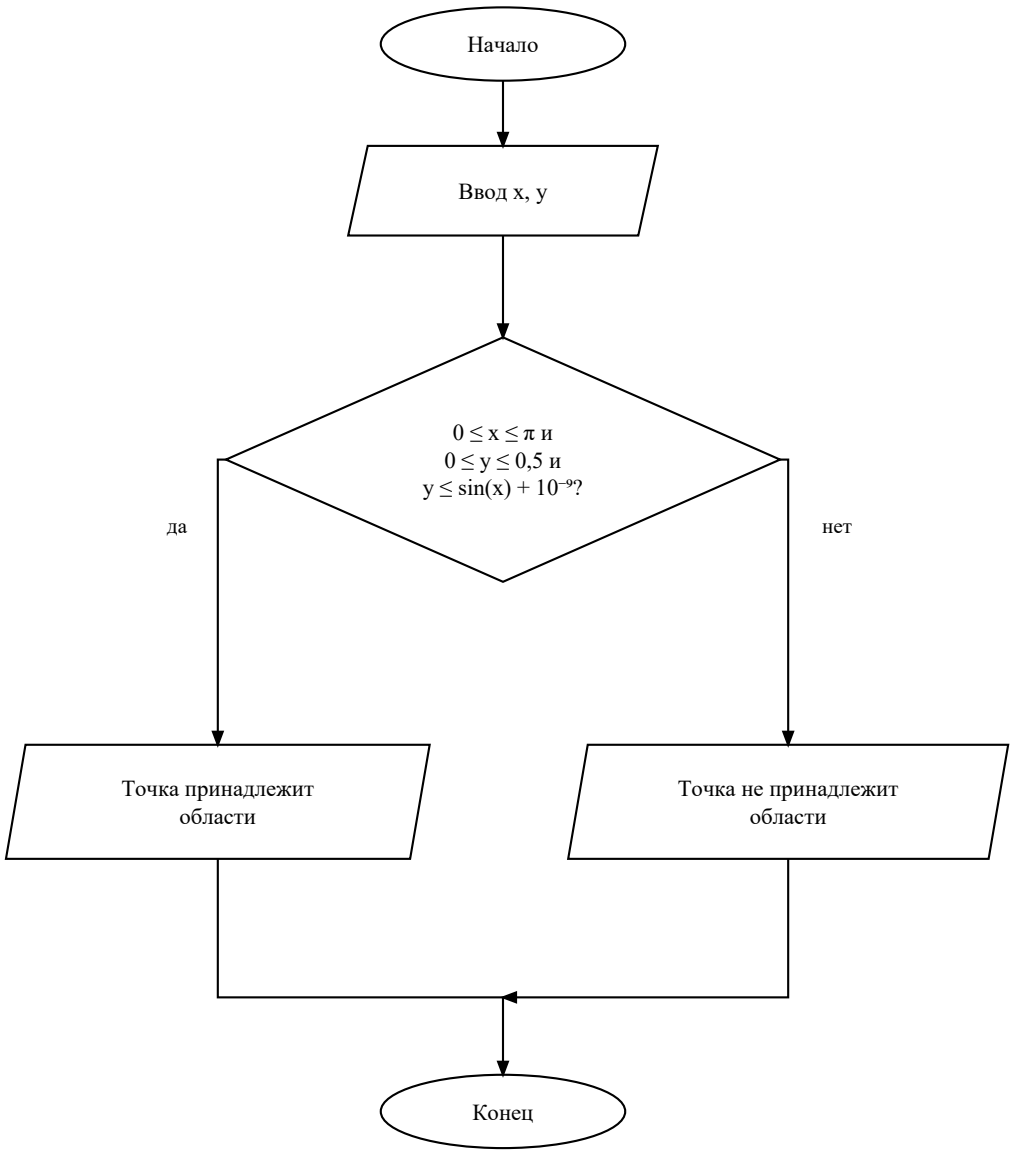


Рисунок 2 - Блок-схема проверки принадлежности точки области

4 ЗАДАНИЕ 3

Условие: треугольник задан длинами сторон a, b, c. Определить, является ли он тупоугольным, прямоугольным или остроугольным. Учесть вырожденные случаи.

Описание алгоритма

Сначала проверяются отрицательные стороны и неравенство треугольника. Нулевая сторона или равенство суммы двух сторон третьей выделяются как вырожденный случай. Затем находится самая длинная сторона L. Для остальных сторон u и v вычисляется разность $q = L^2 - (u^2 + v^2)$. Если q равно нулю, треугольник прямоугольный; если q больше нуля - тупоугольный; иначе - остроугольный. Это следует из теоремы косинусов. Для сравнения с нулем используется допуск 10^{-9}.

Исходный код

```
# Определение вида треугольника по углам
a = float(input("Введите сторону a: "))
b = float(input("Введите сторону b: "))
c = float(input("Введите сторону c: "))

if a < 0 or b < 0 or c < 0:
    print("Стороны не могут быть отрицательными")
elif a + b < c or a + c < b or b + c < a:
    print("Треугольник с такими сторонами не существует")
elif a == 0 or b == 0 or c == 0 or a + b == c or a + c == b or b + c == a:
    print("Вырожденный треугольник: площадь равна нулю")
else:
    # Находим самую длинную сторону
    if a >= b and a >= c:
        difference = a * a - (b * b + c * c)
    elif b >= a and b >= c:
        difference = b * b - (a * a + c * c)
    else:
        difference = c * c - (a * a + b * b)

    if abs(difference) < 0.000000001:
        print("Прямоугольный треугольник")
    elif difference > 0:
        print("Тупоугольный треугольник")
    else:
        print("Остроугольный треугольник")
```

БЛОК-СХЕМА ЗАДАНИЯ 3

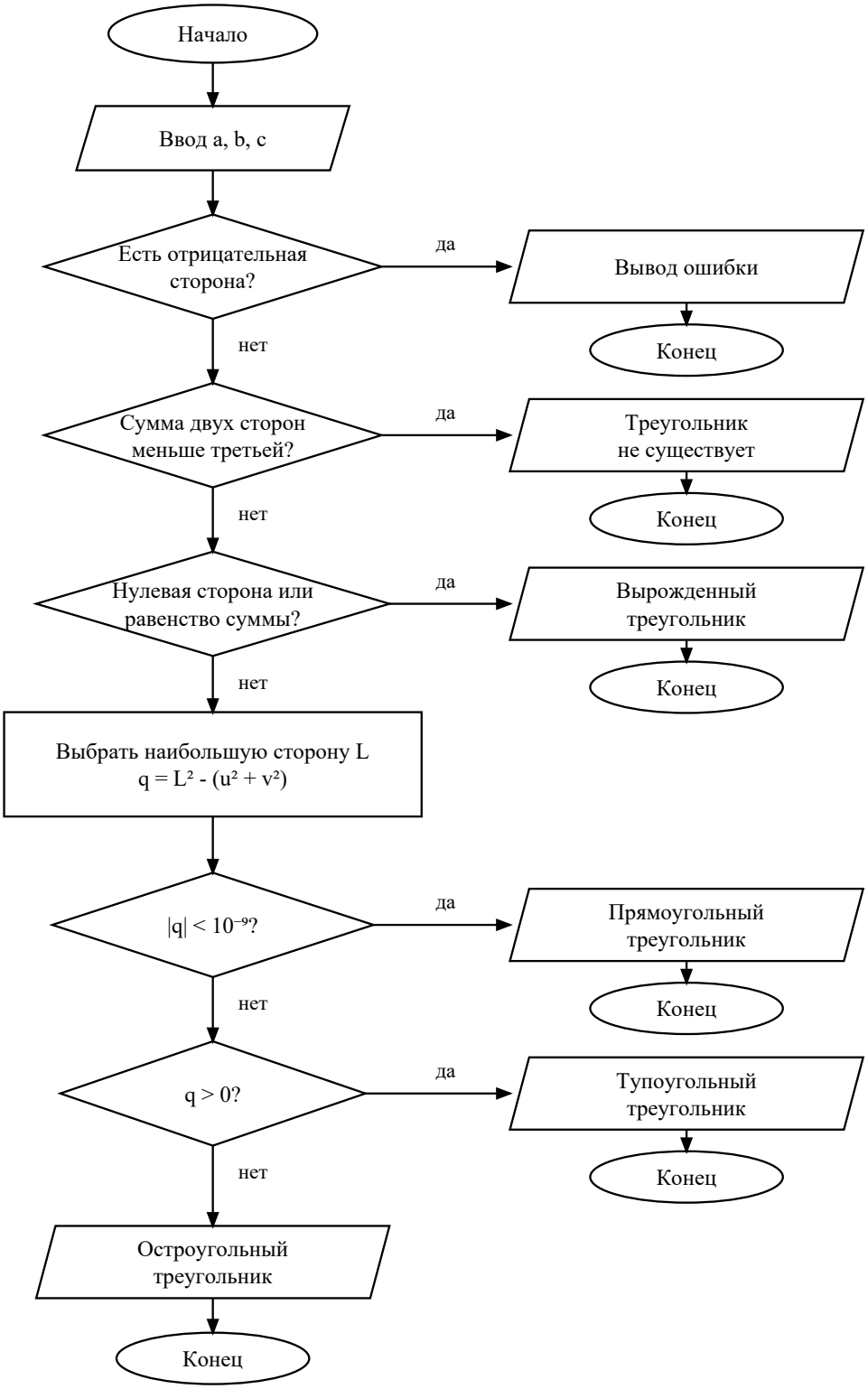


Рисунок 3 - Блок-схема определения вида треугольника

ТЕСТИРОВАНИЕ ЗАДАНИЯ 3

На блок-схеме L - наибольшая из сторон a, b, c; u и v - две оставшиеся стороны. В программе разность q называется difference.

Проверены все три типа треугольника, неверные данные, вырожденные случаи и разные положения наибольшей стороны во входных данных.

a; b; c	Ожидаемый результат	Полученный результат
3; 4; 5	Прямоугольный	Прямоугольный
3; 3; 3	Остроугольный	Остроугольный
2; 3; 4	Тупоугольный	Тупоугольный
1; 2; 4	Не существует	Не существует
1; 2; 3	Вырожденный	Вырожденный
0; 2; 2	Вырожденный	Вырожденный
-1; 2; 2	Ошибка: отрицательная сторона	Стороны не могут быть отрицательными
5; 3; 4	Прямоугольный	Прямоугольный
3; 5; 4	Прямоугольный	Прямоугольный

Результаты работы программы

Для сторон 3, 4 и 5 наибольшая сторона равна 5. Разность составляет 25 - (9 + 16) = 0. Получен прямоугольный треугольник. Для сторон 2, 3 и 4 разность равна 16 - (4 + 9) = 3, поэтому треугольник тупоугольный.

Всего для трех программ выполнено 19 проверок. Во всех случаях полученные результаты совпали с ожидаемыми. Сообщения об ошибках выводятся при нулевом делителе, отрицательных сторонах и нарушении неравенства треугольника.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Для чего используются условные операторы?

Они позволяют выбрать действие в зависимости от условия. Например, можно вычислить выражение при ненулевом делителе или вывести сообщение об ошибке.

2. Как выглядит полная форма ветвления в Python?

Полная форма состоит из if с условием и блока else. При истинном условии выполняется блок if, при ложном - блок else. Для нескольких проверок между ними добавляется elif.

3. Для чего используется else?

Блок else выполняется, когда предшествующие условия if и elif не выполнились. У else собственного условия нет.

4. Какие основные операторы использованы?

Использованы if, elif, else, сравнения ==, <, >, <=, >=, логические and и or, арифметические +, -, *, /. Ввод выполнен через input(), преобразование чисел - float(), вывод - print().

Общий вывод

В ходе работы изучены формы условного оператора if. Составлены блок-схемы и написаны три программы второго варианта. Выполнены вычисление выражения, проверка принадлежности точки области и определение вида треугольника. Работа программ проверена на обычных, граничных и ошибочных данных.